

Reviewer Pack — Quantum Entanglement Index and Read-Twice BP Lower Bounds

Entry 090386d08c4a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 15:19:15 UTC

Quantum Entanglement Index and Read-Twice BP Lower Bounds

Entry ID: 090386d08c4a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 15:19:15 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory **Field B** (complexity object): Branching Program: read-twice (BP)

Statement:

[‘For every read-twice BP P , the quantum entanglement index $E(P)$ is lower-bounded by a polynomial of its size, specifically $\Theta(\log^2(\text{size}(P)))$.’, ‘Equivalently, for any read-twice BP P with entanglement index $E(P) \leq f(n)$, there exists an instance where the BP size exceeds $2^{\{f(n)/\log 2(f(n))\}}$.’, ‘This implies that the entanglement index of a BP provides a non-trivial lower bound on its size.’]

Rationale (proposer’s reasoning):

[‘The quantum entanglement index captures the complexity of distributing a bipartite state, which is related to the difficulty of simulating a read-twice BP. Therefore, if the index is high, it suggests that the BP requires a large number of gates or operations.’, ‘Quantum entanglement invariants have been used successfully in other areas of complexity theory, suggesting their potential applicability to BP lower bounds.’, ‘The polynomial nature of the lower bound implies that the entanglement index provides meaningful information about the BP’s size without overestimating it significantly.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e0208995059664bf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean entanglement index for BPs of size up to 40 is lower-bounded by $\Theta(\log^2(\text{size}))$ with a p-value ≤ 0.05 , using at least 100 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_bp(size):
        bp = [[random.choice([0, 1]) for _ in range(size)] for _ in range(size)]
        return bp

    def compute_entanglement_index(bp):
        size = len(bp)
        index = 0
        for i in range(size):
            for j in range(i + 1, size):
                if bp[i][j] != bp[j][i]:
                    index += 1
        return index

    n = random.randint(5, 40)
    bp = generate_bp(n)
    entanglement_index = compute_entanglement_index(bp)

    metric_name = 'mean_entanglement_index'
    metric_value = entanglement_index
    instances_tested = 1
    conjecture_holds = False
    counterexample = "mapping_undefined"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
```

```

        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_entanglement_index = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_entanglement_index} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_entanglement_index} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ined'}
TRIAL: {'metric_name': 'mean_entanglement_index', 'metric_value': 10, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'mean_entanglement_index', 'metric_value': 133, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'mean_entanglement_index', 'metric_value': 115, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'mean_entanglement_index', 'metric_value': 24, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'mean_entanglement_index', 'metric_value': 341, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'mean_entanglement_index', 'metric_value': 59, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'mean_entanglement_index', 'metric_value': 336, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'mean_entanglement_index', 'metric_value': 78, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'mean_entanglement_index', 'metric_value': 61, 'instances_tested': 1, 'conjecture_holds': True}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_182b86e8.py", line 69, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_182b86e8.py", line 69, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15435
2	preregistration	ollama_remote	glm4:latest	0	0	9500
3	novelty	ollama_remote	glm4:latest	0	0	8781
4	novelty	ollama_remote	glm4:latest	0	0	8944
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19487
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7313
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8490
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6639
9	judge	ollama_remote	glm4:latest	0	0	11647

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 96236 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/090386d08c4a.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/090386d08c4a.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/090386d08c4a.tar.gz* (if generated)